

ALGORITMA DAN STRUKTUR DATA
LATIHAN 1 REVIEW ARRAY, POINTER DAN STRUKTUR



Dosen Pembimbing:

Prof. Dr Arna Fariza S.Kom., M.Kom

Disusun oleh:

Nama: Luthfi Bahrur Rozaq

NRP: 3125600069

1 D4 TEKNIK INFORMATIKA C
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

LATIHAN

1. Terdapat array dengan tipe char yang berisi {'p', 'e', 'n', 's', 'i', 't'}, baliklah array

```
1. #include <stdio.h>
2. #include <string.h>
3.
4. int main() {
5.     char arr[] = "pensit";
6.     int pjg = strlen(arr);
7.
8.     printf("kata awal : %s\n\n", arr);
9.
10.    for (int i = 0; i < pjg / 2; i++)
11.    {
12.        char tmp = arr[i];
13.        arr[i] = arr[pjg - i - 1];
14.        arr[pjg - i - 1] = tmp;
15.    }
16.
17.    printf("setelah dibalik : %s", arr);
18.    return 0;
19.}
```

Output:

```
kata awal : pensit

setelah dibalik : tisnep
```

Analisis Program:

Pada bahasa C, data array disimpan dalam memori yang berurutan, dengan elemen pertama berindeks 0. Untuk mencetak array secara terbalik tanpa mengubah struktur datanya, digunakan perulangan for yang berjalan mundur. Variabel iterator i dimulai dari indeks maksimal array (panjang array dikurangi 1), lalu berkurang (i--) hingga mencapai indeks 0.

2. Terdapat dua buah matrik dengan ordo n yang direpresentasikan dengan array dimensi dua, lakukan operasi penjumlahan, pengurangan dan perkalian.

```
1. #include <stdio.h>
2. #define max 10
```

```

3.
4. void jumlah(int n, int a[max][max], int b[max][max], int
   c[max][max]);
5. void kurang(int n, int a[max][max], int b[max][max], int
   c[max][max]);
6. void kali(int n, int a[max][max], int b[max][max], int
   c[max][max]);
7.
8. int m1[max][max];
9. int m2[max][max];
10. int hasil[max][max];
11.
12. int main()
13. {
14.     int n, i, j;
15.
16.     printf("masukkan ordo matriks (contoh: 2 untuk 2x2): ");
17.     scanf("%d", &n);
18.
19.     printf("masukkan matrix pertama:\n");
20.     for (i = 0; i < n; i++)
21.     {
22.         for (j = 0; j < n; j++)
23.         {
24.             scanf("%d", &m1[i][j]);
25.         }
26.     }
27.
28.     printf("masukkan matrix kedua:\n");
29.     for (i = 0; i < n; i++)
30.     {
31.         for (j = 0; j < n; j++)
32.         {
33.             scanf("%d", &m2[i][j]);
34.         }
35.     }
36.
37.     jumlah(n, m1, m2, hasil);
38.
39.     printf("\nhasil penjumlahan:\n");
40.     for (i = 0; i < n; i++)

```

```

41.     {
42.         for (j = 0; j < n; j++)
43.         {
44.             printf("%d ", hasil[i][j]);
45.         }
46.         printf("\n");
47.     }
48.
49.     kurang(n, m1, m2, hasil);
50.
51.     printf("\nhasil pengurangan:\n");
52.     for (i = 0; i < n; i++)
53.     {
54.         for (j = 0; j < n; j++)
55.         {
56.             printf("%d ", hasil[i][j]);
57.         }
58.         printf("\n");
59.     }
60.
61.     kali(n, m1, m2, hasil);
62.
63.     printf("\nhasil perkalian:\n");
64.     for (i = 0; i < n; i++)
65.     {
66.         for (j = 0; j < n; j++)
67.         {
68.             printf("%d ", hasil[i][j]);
69.         }
70.         printf("\n");
71.     }
72.
73.     return 0;
74. }
75.
76. void jumlah(int n, int a[max][max], int b[max][max], int
    c[max][max])
77. {
78.     int i, j;
79.     for (i = 0; i < n; i++)
80.     {

```

```

81.         for (j = 0; j < n; j++)
82.         {
83.             c[i][j] = a[i][j] + b[i][j];
84.         }
85.     }
86.}
87.
88.void kurang(int n, int a[max][max], int b[max][max], int
    c[max][max])
89.{
90.    int i, j;
91.    for (i = 0; i < n; i++)
92.    {
93.        for (j = 0; j < n; j++)
94.        {
95.            c[i][j] = a[i][j] - b[i][j];
96.        }
97.    }
98.}
99.
100. void kali(int n, int a[max][max], int b[max][max], int
    c[max][max])
101. {
102.     int i, j, k;
103.     for (i = 0; i < n; i++) {
104.         for (j = 0; j < n; j++) {
105.             c[i][j] = 0;
106.             for (k = 0; k < n; k++) {
107.                 c[i][j] += a[i][k] * b[k][j];
108.             }
109.         }
110.     }
111. }

```

Output:

```
masukkan ordo matriks (contoh: 2 untuk 2x2): 2
masukkan matrix pertama:
1
2
3
4
masukkan matrix kedua:
5
6
7
8

hasil penjumlahan:
6 8
10 12

hasil pengurangan:
-4 -4
-4 -4

hasil perkalian:
19 22
43 50
```

Analisis Program:

Matriks direpresentasikan menggunakan array dua dimensi (baris dan kolom). Operasi penjumlahan dan pengurangan dilakukan dengan menjumlahkan elemen pada indeks baris dan kolom yang sama secara langsung. Untuk perkalian matriks, diperlukan 3 perulangan bersarang (nested loop) karena operasi yang dilakukan adalah perkalian baris matriks pertama dengan kolom matriks kedua yang diakumulasikan.

3. Bagaimana output program di bawah ini ?

```
1. #include <stdio.h>
2.
3. int main() {
4.     int count = 10, *temp, sum = 7;
5.
6.     temp = &count;
7.     *temp = 32;
8.     temp = &sum;
9.     *temp = count;
10.    sum = *temp * 4;
11.
12.    printf("count=%d, *temp=%d, sum=%d\n", count, *temp,
           sum);
13. }
```

Output:

```
count=32, *temp=128, sum=128
```

Analisis Tracing Variabel:

1. count = 10, sum = 7 (Nilai awal)
2. temp = &count -> Pointer temp menyimpan alamat dari count.
3. *temp = 32 -> Nilai yang ditunjuk oleh temp (yaitu count) diubah menjadi 32. Sekarang, count = 32.
4. temp = &sum -> Pointer temp pindah menunjuk ke alamat sum.
5. *temp = count -> Nilai yang ditunjuk temp (yaitu sum) diubah menjadi nilai count saat ini (32). Sekarang sum = 32.
6. sum = *temp * 4 -> sum = 32 * 4. Nilai sum diperbarui menjadi 128.
7. Saat mencetak layar, count bernilai 32, *temp (yang sedang menunjuk ke sum) bernilai 128, dan sum bernilai 128.

4. Masalah aritmatika polinom adalah membuat sekumpulan subrutin manipulasi terhadap polinom simbolis (symbolic Polynomial). Misalnya:

$$P1 = 6x^8 + 8x^7 + 5x^5 + x^3 + 15$$

$$P2 = 3x^9 + 4x^7 + 3x^4 + 2x^3 + 2x^2 + 10P3 = x^2 + 5$$

Terdapat empat operasi aritmatika polinom dasar antara lain:

- a. Penambahan ($P1 + P2 = 3x^9 + 6x^8 + 12x^7 + 5x^5 + 3x^4 + 3x^3 + 2x^2 + 25$)
- b. Pengurangan ($P1 - P2 = -3x^9 + 6x^8 + 4x^7 + 5x^5 - 3x^4 - x^3 - 2x^2 + 5$)
- c. Perkalian ($P1 * P3 = 6x^{10} + 8x^9 + 5x^7 + x^5 + 15x^2 + 30x^8 + 40x^7 + 25x^5 + 5x^3 + 75 = 6x^{10} + 8x^9 + 30x^8 + 45x^7 + 26x^5 + 5x^3 + 15x^2 + 75$)
- d. Turunan ($P2' = 27x^8 + 28x^6 + 12x^3 + 6x^2 + 4x$)

Representasikan bilangan polinom dengan array dan buatlah prosedur-prosedur yang melakukan kelima operasi aritmatika di atas.

```
1. #include <stdio.h>
2.
3. void tambah (int [], int [], int [], int, int, int);
4. void kurang (int [], int [], int [], int, int, int);
5. void kali (int [], int [], int [], int, int, int);
6. void turunan (int [], int [], int);
7.
8. int main() {
9.     int p1[] = {15, 0, 0, 1, 0, 5, 0, 8, 6};
10.    int p2[] = {10, 0, 2, 2, 3, 0, 0, 4, 0, 3};
11.    int p3[] = {5, 0, 1};
12.    int batas_p1 = sizeof(p1) / sizeof(p1[0]);
13.    int batas_p2 = sizeof(p2) / sizeof(p2[0]);
14.    int batas_p3 = sizeof(p3) / sizeof(p3[0]);
```

```

15.     int batasmaks, i;
16.
17.     if (batas_p1 > batas_p2) {
18.         batasmaks = batas_p1;
19.     } else {
20.         batasmaks = batas_p2;
21.     }
22.     int hasiltbh[batasmaks];
23.     int hasilkrgr[batasmaks];
24.     int hasilkali[batas_p1 + batas_p3 - 1];
25.     int hasiltrn[batas_p2 - 1];
26.
27.     tambah(p1, p2, hasiltbh, batas_p1, batas_p2, batasmaks);
28.     kurang(p1, p2, hasilkrgr, batas_p1, batas_p2, batasmaks);
29.     kali(p1, p3, hasilkali, batas_p1, batas_p3, batas_p1 +
    batas_p3 - 1);
30.     turunan(p2, hasiltrn, batas_p2);
31.
32.     printf ("Hasil dari P1 + P2 = ");
33.     for (i = 0; i < batasmaks; i++)
34.     {
35.         printf ("%d ", hasiltbh[i]);
36.     }
37.     puts ("");
38.     printf ("Hasil dari P1 - P2 = ");
39.     for (i = 0; i < batasmaks; i++)
40.     {
41.         printf ("%d ", hasilkrgr[i]);
42.     }
43.     puts ("");
44.     printf ("Hasil dari P1 * P3 = ");
45.     for (i = 0; i < batas_p1 + batas_p3 - 1; i++)
46.     {
47.         printf ("%d ", hasilkali[i]);
48.     }
49.     puts ("");
50.     printf ("Hasil dari P2' = ");
51.     for (i = 0; i < batas_p2 - 1; i++)
52.     {
53.         printf ("%d ", hasiltrn[i]);
54.     }

```



```

55.     puts ("");
56.}
57.void tambah(int satu[], int dua[], int hasil[], int batas1,
    int batas2, int batasmaks) {
58.    int i;
59.    for (i = 0; i < batasmaks ; i++) {
60.        hasil[i] = 0;
61.    }
62.    for (i = 0; i < batasmaks; i++) {
63.        int nilai1 = 0, nilai2 = 0;
64.        if (i < batas1)
65.        {
66.            nilai1 = satu[i];
67.        }
68.        if (i < batas2)
69.        {
70.            nilai2 = dua[i];
71.        }
72.        hasil[i] = nilai1 + nilai2;
73.    }
74.}
75.void kurang(int satu[], int dua[], int hasil[], int batas1,
    int batas2, int batasmaks) {
76.    int i;
77.
78.    for (i = 0; i < batasmaks; i++) {
79.        hasil[i] = 0;
80.    }
81.    for (i = 0; i < batasmaks; i++) {
82.        int nilai1 = 0, nilai2 = 0;
83.        if (i < batas1)
84.        {
85.            nilai1 = satu[i];
86.        }
87.        if (i < batas2)
88.        {
89.            nilai2 = dua[i];
90.        }
91.        hasil[i] = nilai1 - nilai2;
92.    }
93.}

```

```

94. void kali(int satu[], int tiga[], int hasil[], int batas1,
    int batas3, int batasmaks) {
95.     int i, j;
96.
97.     for (i = 0; i < batasmaks; i++)
98.     {
99.         hasil[i] = 0;
100.    }
101.    for (i = 0; i < batas1; i++)
102.    {
103.        for (j = 0; j < batas3; j++)
104.        {
105.            hasil[i + j] += (satu[i] * tiga[j]);
106.        }
107.    }
108.    }
109. }
110. void turunan(int dua[], int hasil[], int batas2) {
111.     int i;
112.
113.     for (i = 1; i < batas2; i++)
114.     {
115.         hasil[i - 1] = dua[i] * i;
116.     }
117. }

```

Output:

```

Hasil dari P1 + P2 = 25 0 2 3 3 5 0 12 6 3
Hasil dari P1 - P2 = 5 0 -2 -1 -3 5 0 4 6 -3
Hasil dari P1 * P3 = 75 0 15 5 0 26 0 45 30 8 6
Hasil dari P2' = 0 4 6 12 0 0 28 0 27

```

Analisis Program:

Polinomial direpresentasikan ke dalam array 1 dimensi dimana indeks array berfungsi sebagai representasi nilai pangkat (eksponen), sedangkan nilai/isi elemen array merupakan koefisien dari suku tersebut. Penggunaan subrutin (fungsi/prosedur) membantu mengisolasi logika tiap operasi agar lebih terstruktur. Pada proses perkalian, digunakan dua perulangan bersarang untuk melakukan sifat distributif, dan operator penambahan majemuk (+=) digunakan untuk mengakumulasi nilai pada indeks (pangkat) yang sama.

5. Bilangan kompleks berbentuk $a + bi$, dimana a dan b adalah bilangan nyata dan $i^2 = -1$. Terdapat empat operasi aritmatika dasar untuk bilangan kompleks, yaitu:

- Penambahan : $(a+bi) + (c+di) = (a+c) + (b+d)i$
- Pengurangan : $(a+bi) - (c+di) = (a-c) + (b-d)i$
- Perkalian : $(a+bi) * (c+di) = (ac-bd) + (ad+bc)i$
- Pembagian : $(a+bi) / (c+di) = [(ac+bd) / (a^2 + b^2)] + [(bc-ad)/(c^2 + d^2)]i$

Tulis program yang membaca dua bilangan kompleks dan simbol operasi yang perlu dilakukan, kemudian lakukan operasi yang diminta. Gunakan struktur untuk merepresentasikan bilangan kompleks dan gunakan prosedur untuk implementasi tiap operasi.

```
1. #include <stdio.h>
2. #include <math.h>
3.
4. typedef struct {
5.     float real;
6.     float imag;
7. } kompleks;
8.
9. kompleks tambah(kompleks k1, kompleks k2);
10. kompleks kurang(kompleks k1, kompleks k2);
11. kompleks kali(kompleks k1, kompleks k2);
12. kompleks bagi(kompleks k1, kompleks k2);
13.
14. int main() {
15.     kompleks k1, k2, hasil;
16.     char ops;
17.
18.     printf("Masukkan nilai real dan imajiner pertama [real,
        imajiner] : ");
19.     scanf("%f %f", &k1.real, &k1.imag);
20.
21.     printf("Masukkan nilai real dan imajiner kedua [real,
        imajiner] : ");
22.     scanf("%f %f", &k2.real, &k2.imag);
23.
24.     printf("Ingin anda apakan dua bagian itu [+ , - , * , /] :
        ");
25.     scanf(" %c", &ops);
```

```

26.
27.     switch (ops) {
28.         case '+':
29.             hasil = tambah(k1, k2);
30.             printf("Hasil: %.2f + %.2fi\n", hasil.real,
                hasil.imag);
31.             break;
32.         case '-':
33.             hasil = kurang(k1, k2);
34.             printf("Hasil: %.2f + %.2fi\n", hasil.real,
                hasil.imag);
35.             break;
36.         case '*':
37.             hasil = kali(k1, k2);
38.             printf("Hasil: %.2f + %.2fi\n", hasil.real,
                hasil.imag);
39.             break;
40.         case '/':
41.             hasil = bagi(k1, k2);
42.             printf("Hasil: %.2f + %.2fi\n", hasil.real,
                hasil.imag);
43.             break;
44.         default:
45.             printf("Error : operator tidak diketahui\n");
46.             break;
47.     }
48.}
49.kompleks tambah(kompleks k1, kompleks k2) {
50.    kompleks hasil;
51.    hasil.real = k1.real + k2.real;
52.    hasil.imag = k1.imag + k2.imag;
53.    return hasil;
54.}
55.kompleks kurang(kompleks k1, kompleks k2) {
56.    kompleks hasil;
57.    hasil.real = k1.real - k2.real;
58.    hasil.imag = k1.imag - k2.imag;
59.    return hasil;
60.}
61.kompleks kali(kompleks k1, kompleks k2) {
62.    kompleks hasil;

```

```

63.     hasil.real = (k1.real * k2.real) - (k1.imag * k2.imag);
64.     hasil.imag = (k1.real * k2.imag) + (k1.imag * k2.real);
65.     return hasil;
66. }
67. kompleks bagi(kompleks k1, kompleks k2) {
68.     kompleks hasil;
69.     float penyebut = pow(k2.real, 2) + pow(k2.imag, 2);
70.
71.     hasil.real = ((k1.real * k2.real) + (k1.imag * k2.imag))
        / penyebut;
72.     hasil.imag = ((k1.imag * k2.real) - (k1.real * k2.imag))
        / penyebut;
73.     return hasil;
74. }

```

Output:

```

Masukkan nilai real dan imajiner pertama [real, imajiner] : 4, 2
Masukkan nilai real dan imajiner kedua [real, imajiner] : 1, 3
Ingin anda apakan dua bagian itu [+ , - , * , /] : +
Hasil: 5.00 + 5.00i
Masukkan nilai real dan imajiner pertama [real, imajiner] : 5, 6
Masukkan nilai real dan imajiner kedua [real, imajiner] : 2, 4
Ingin anda apakan dua bagian itu [+ , - , * , /] : -
Hasil: 3.00 + 2.00i
Masukkan nilai real dan imajiner pertama [real, imajiner] : 3, 2
Masukkan nilai real dan imajiner kedua [real, imajiner] : 1, 4
Ingin anda apakan dua bagian itu [+ , - , * , /] : *
Hasil: -5.00 + 14.00i
Masukkan nilai real dan imajiner pertama [real, imajiner] : 4, 2
Masukkan nilai real dan imajiner kedua [real, imajiner] : 1, 1
Ingin anda apakan dua bagian itu [+ , - , * , /] : /
Hasil: 3.00 + -1.00i

```

Analisis Program:

Bilangan kompleks secara logis memiliki dua entitas yang tak terpisahkan: bagian riil dan imajiner. Konsep struct sangat tepat digunakan untuk mengelompokkan kedua variabel bertipe float ini ke dalam satu kesatuan tipe data baru. Bahasa C melewati struktur ke dalam fungsi dengan konsep pass-by-value (menyalin isi). Oleh karena itu, prosedur perhitungan diimplementasikan untuk mengembalikan (return) sebuah data struktur bilangan kompleks baru ke program utama, alih-alih menggunakan subrutin bertipe void.

Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Array adalah struktur data fundamental untuk menyimpan kumpulan elemen dengan tipe seragam di blok memori yang berurutan. Representasi indeks sangat berguna untuk memodelkan struktur matematika kompleks seperti polinomial.
2. Pointer adalah variabel yang menyimpan alamat memori dari variabel lain. Kombinasi pointer memungkinkan manipulasi memori secara langsung dan efisien, meskipun membutuhkan kehati-hatian agar tidak terjadi kebocoran memori.
3. Struktur (Struct) memungkinkan pengelompokan beberapa variabel dengan tipe data yang berbeda ke dalam satu entitas, yang sangat berguna untuk membungkus data multi-atribut seperti bilangan kompleks.
4. Secara konseptual, array berhubungan erat dengan alamat memori sekuensial, sedangkan struct lebih difokuskan pada pengorganisasian data heterogen.